

Spatial and temporal errors

Memory corruption errors are sometimes classified as:

- ► spatial: the error happens because memory access exceeds some region of memory that a data item is intended to occupy.
 - **main example:** buffer overflow
 - **This lecture focuses on spatial errors!**
- ► temporal: the error happens because memory access occurs in some region of memory that the program does not currently have access to
 - **main example:** dangling pointer

Buffer overflow

Buffer overflow is a common vulnerability.

Simple cause:

- putting **m** bytes into a buffer of size **n**, for **m > n**
- corrupts the surrounding memory

Simple fix:

- check size of data before/when writing

Overflow-type exploits, where corruption performs something specific the attacker wants, can be very complex.

We'll study examples to explain how devastating overflows can be, looking at simple (and historical) stack overflows and heap overflows.

Examples will use Linux/x86 to demonstrate; principles are similar on other OSes/architectures.

Corrupting stack variables

Local variables are put close together on the stack.

- ▶ If a stray write goes beyond the size of one variable
- ▶ ... it can corrupt another

Application scenario

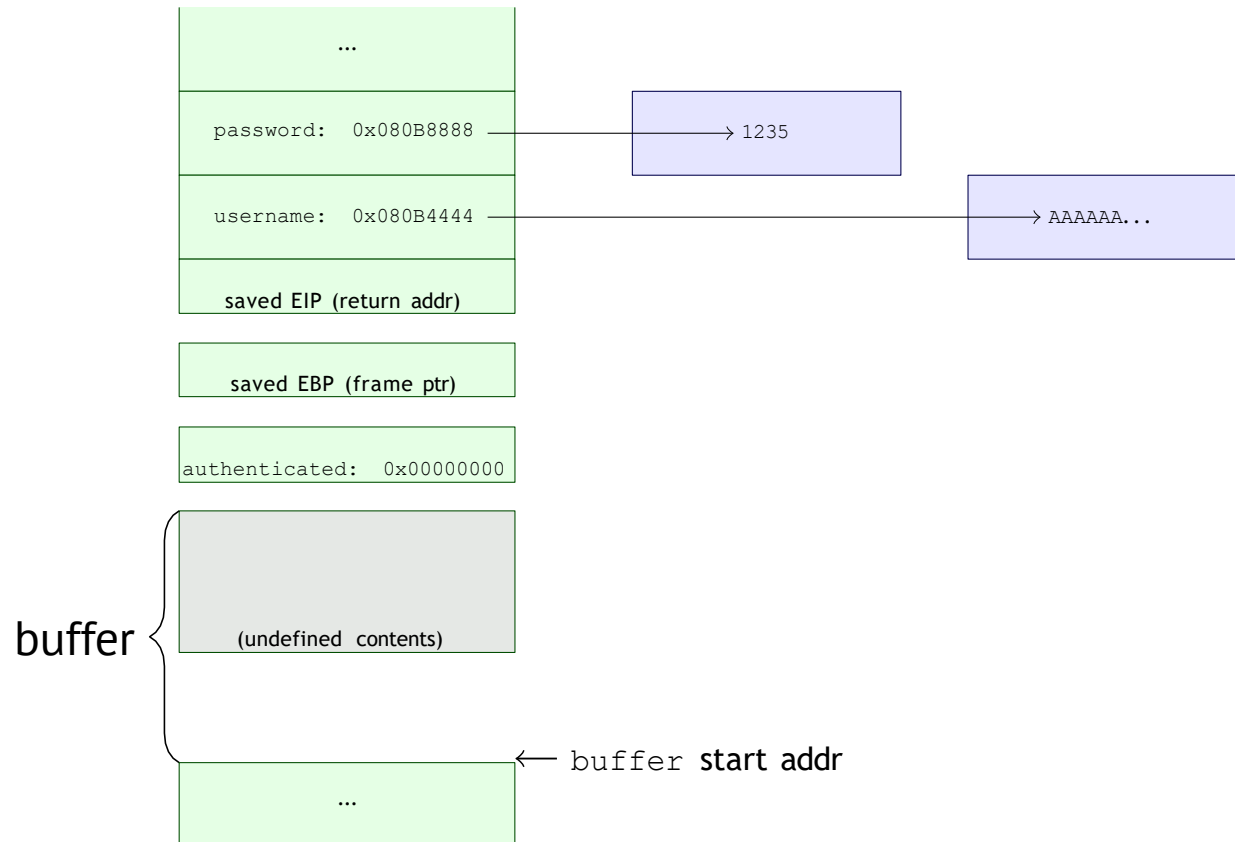
```
int authenticate(char *username, char *password) {  
  
    int authenticated; // flag, non-zero if authenticated  
    char buffer[1024]; // buffer for log message  
  
    authenticated = verify_password(username, password);  
    if (authenticated == 0) {  
        sprintf(buffer, "Incorrect password for user %s\n", username);  
        log("%s", buffer);  
    }  
  
    return authenticated;  
}
```

Application scenario

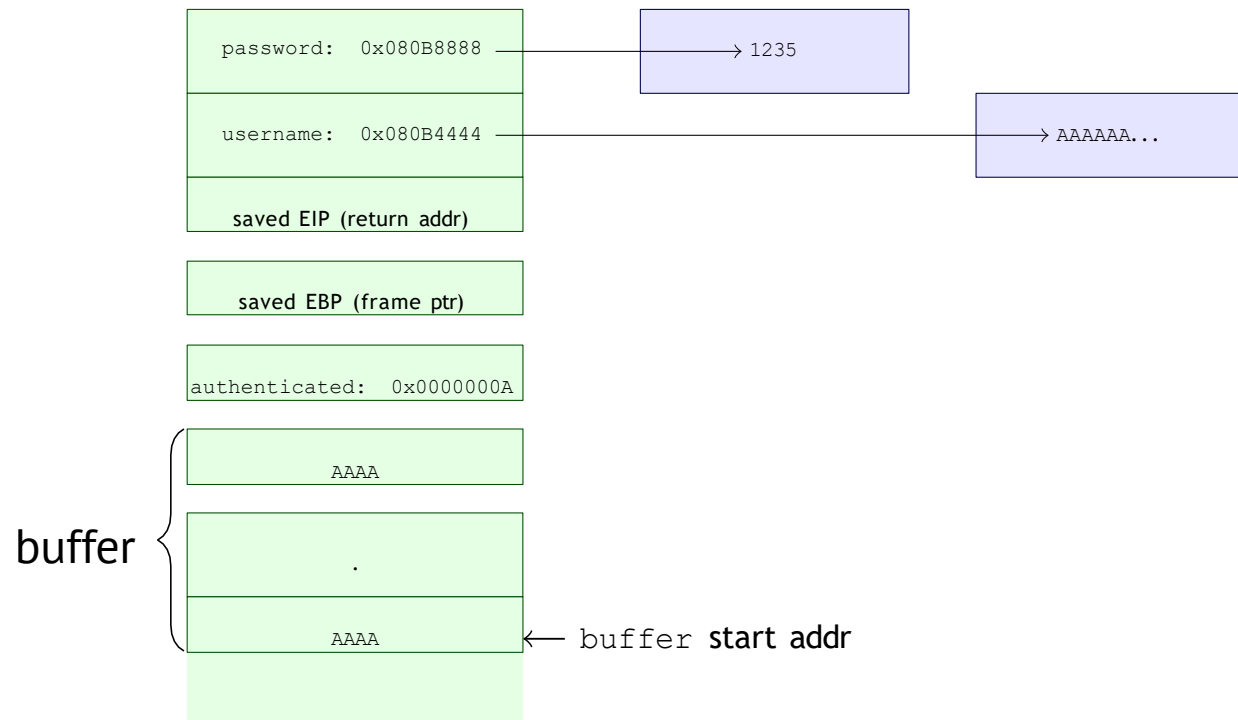
```
int authenticate(char *username, char *password) {  
  
    int authenticated; // flag, non-zero if authenticated  
    char buffer[1024]; // buffer for log message  
  
    authenticated = verify_password(username, password);  
    if (authenticated == 0) {  
        sprintf(buffer, "Incorrect password for user %s\n", username);  
        log("%s", buffer);  
    }  
  
    return authenticated;  
}
```

- ▶ Vulnerability in **authenticate()** call to **sprintf()**.
- ▶ If the username is longer than **995 bytes**, data will be written past the end of the buffer.

Possible stack frame before exploit



Stack frame after exploit



- ▶ If *username* is >995 letters long, *authenticated* is corrupted and may be set to non-zero.
- ▶ E.g., char 1024='\\n', the low byte becomes 10.

Local variable corruption remarks

► Tricky in practice:

- location of variables may not be known
- memory addresses can vary between invocations
- C standards don't specify stack layout
- compiler moves things around, optimizes layout
- effect depends on behavior of application code

► A more predictable, general attack works by corrupting the fixed information in every stack frame: the frame pointer and return address.

Classic stack overflow exploit



- The malicious argument overwrites all of the space allocated for the buffer, all the way to the return address location.
- The return address is altered to point back into the stack, somewhere before the attack code. Typically, the attack code executes a shell.

Attacker controlled execution

By over-writing the return address, the attacker may either:

1. set it to point to some known piece of the application code, or code inside a shared library, which achieves something useful, or
2. supply his/her own code somewhere in memory, which may do anything, and arrange to call that.

The second option is the most general and powerful. How does it work?

Arbitrary code exploit

- ▶ The attacker takes these steps:
 1. write code useful for an attacker
 2. store executable code somewhere in memory
 3. use a stack overflow to re-direct execution there

- ▶ The attack code is known as shellcode. Typically, it launches a shell or network connection.

- ▶ **Shellcode is ideally:**
 - small and self-contained
 - position independent
 - free of ASCII NUL (0x00) characters

Arbitrary code exploit

- 1. write code useful for an attack**
2. store executable code somewhere in memory
3. use a stack overflow to re-direct execution there

Building shellcode

► When spawning a shell in Unix. The code looks like this:

```
#include <unistd.h>
...
char *args[] = { "/bin/sh", NULL };
execve("bin/sh", args, NULL)
```

- `execve()` is part of the Standard C Library, `libc` (`stdlib.h`)
- it starts a process with the given name and argument list and the environment as the third parameter.

We want to write (relocatable) assembly code which does the same thing: constructing the argument lists and then invoking the `execve()` function.

Invoking system calls

- ▶ To execute a library function, the code would need to find the location of the function.
 - for a dynamically loaded library, this requires ensuring it is loaded into memory, negotiating with the linker
 - this would need quite a bit of assembly code
- ▶ It is easier to make a system call directly to the operating system.
And, `execve()` is a library call which implements a system call!

Invoking system calls

► Reminder: Linux system calls (32bit x86) operate like this:

- Store parameters in registers EBX, ECX, ...
- Put the desired system call number into EAX
- Use the interrupt **int 128 (0x80)** to trigger the call

Invoking a shell

Here is the assembly code for a simple system call invoking a shell:

```
.section .rodata                ; data section

args:
    dd arg                      ; Define double word (4 bytes) containing the address of 'arg'
    dd 0                        ; Define double word (4 bytes) containing the value 0

arg:
    db "/bin/sh", 0             ; Define bytes for the string "/bin/sh" with a null terminator

    .text
    .globl main                 ; Make 'main' globally visible

main:
    mov ebx, offset arg         ; Move the address of 'arg' into EBX
    mov ecx, offset args        ; Move the address of 'args' into ECX
    mov edx, 0                  ; Move the immediate value 0 into EDX
    mov eax, 0xB                ; Move the immediate value 0xB (11 decimal) into EAX
    int 0x80                    ; Execute interrupt 0x80 => #execve("/bin/sh", ["/bin/sh"], NULL)
    ret
```


From assembly to shellcode

However, this is not yet quite shellcode: it contains hard-wired (absolute) addresses and a data section.

Question. How do we turn this into position independent code without separate data?

From assembly to shellcode

- Moreover, we need to find the binary representation of the instructions (i.e., the compiled shell code).
- This will be the data that we can then feed back into our attack.

\$ gcc shellcode.s -o shellcode.out

```
$ objdump -d shellcode.out
...
080483ed <main>:
 80483ed:    bb a8 84 04 08          mov     $0x80484a8,%ebx
 80483f2:    b9 a0 84 04 08          mov     $0x80484a0,%ecx
 80483f7:    ba 00 00 00 00          mov     $0x0,%edx
 80483fc:    b8 0b 00 00 00          mov     $0xb,%eax
 8048401:    cd 80                  int     $0x80
 8048403:    c3                     ret
```

- We take the hex op code sequence `bb a8 84...` and encode it as a string (or URL, filename, etc.) **to feed into the program as malicious input.**
- There's a bit of an art to crafting shellcode for different architectures and scenarios. Handily many examples are online.
 - For example, at:
 - shell-storm.org/shellcode
 - www.exploit-db.com/shellcodes.

Arbitrary code exploit

1. write code useful for an attack
2. **store executable code somewhere in memory**
3. **use a stack overflow to re-direct execution there**

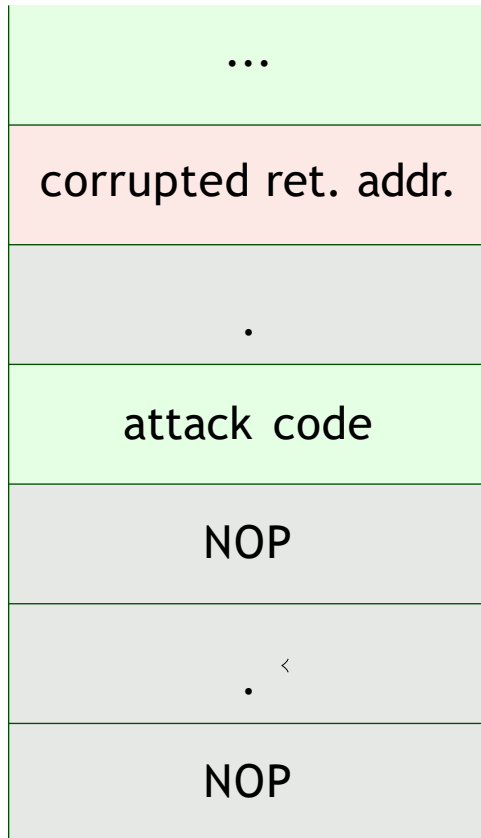
► Two options:

- shellcode on stack
- shellcode in another part of the program data

► Problem in both cases is :

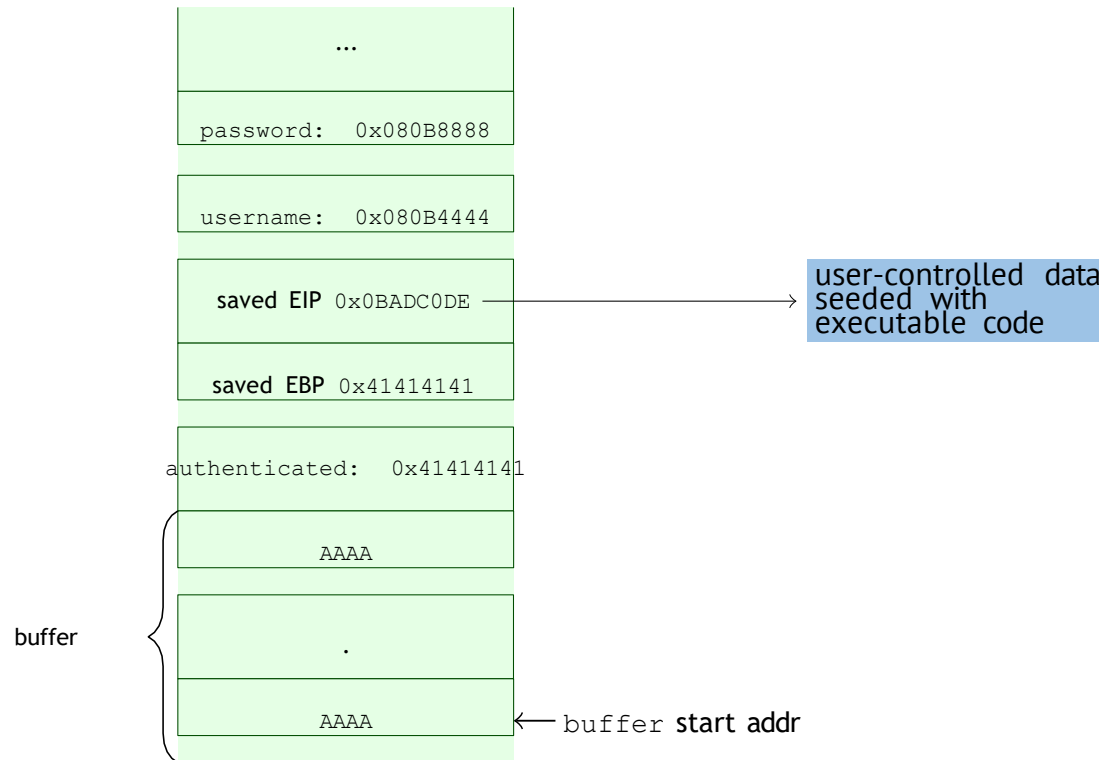
- how to find out where the code is?

Attack code on stack: the NOP sled



- The exact address of the attack code in the stack is hard to guess. The attacker can increase the chance of success by allowing a range of addresses to work.
- The overflow uses a **NOP sled**, which the CPU execution "lands on", before being directed to the attack code.

Attack code elsewhere in memory



- Various (sometimes intricate) possibilities
- ... in an environment variable, modifying function pointers, corrupting caller's saved frame pointer

Stack smashing without shellcode

Sometimes an attacker cannot directly inject code which gets executed but can still corrupt return addresses.

- **Return to library (ret2libc):** The attacker overflows a buffer causing the return instruction to jump to invoke `system()` with an argument pointing to `/bin/sh`.
- **Return-oriented Programming (ROP):** Sequences of instructions (gadgets) from library code are assembled together to manipulate registers, eventually to invoke a library function or even to make a Turing-complete language.

Canaries on the stack

Each stack frame includes vulnerable location pointers which may be corrupted in a stack overflow attack.

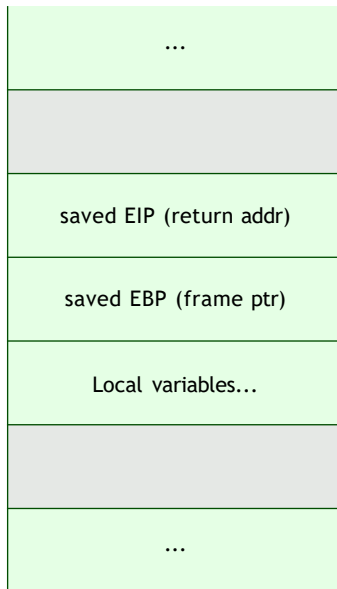
► Idea:

- wrap frame with protective layer, a “canary”
- canary sits below return address
- attacker overflows stack buffer to hit return address
 - necessarily overwrites canary

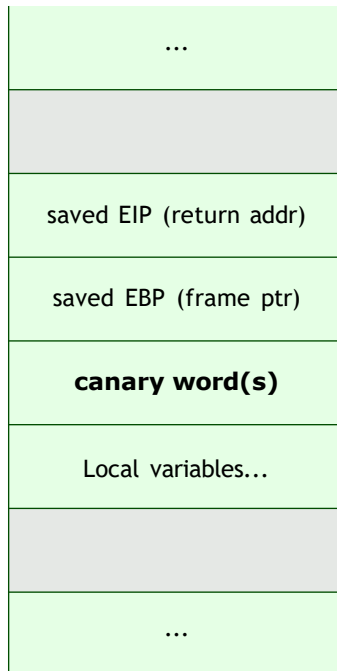
► generated code adds and checks canaries

Early proposal: [StackGuard](#) compiler.

Stack without canaries



Stack with canary



- The "canary" is special data written into the stack to detect unexpected modifications.
- If a stack overflow or other corruption occurs, the canary may be altered. The compiler adds extra instructions to insert canaries and check their integrity.

GCC's Stack Smashing Protector

Consider this C program:

```
#include <stdio.h>
#include <string.h>

int fun1(char *arg) {
    char buffer[1024];
    strcpy(buffer, arg);
}

void main(int argc, char *argv[]) {
    fun1(argv[1]);
}
```

Let's compare the assembler compiled with:

1. `gcc -S -m32 -fno-stack-protector`
2. `gcc -S -m32`

gcc -S -m32 -fno-stack-protector

```
main: ; code without SSP: gcc -S -m32 -fno-stack-protector
```

```
    push ebp
    mov ebp, esp
    and esp, -16           ; align stack to 16-byte
    sub esp, 16
    mov eax, [ebp + 12]    ; eax = addr of argv
    add eax, 4             ; eax = addr of argv[1]
    mov eax, [eax]         ; eax = contents of argv[1]
    mov [esp], eax        ; push it
    call fun1
    leave
    ret
```

```
fun1:
```

```
    push ebp              ; save old frame ptr
    mov ebp, esp          ; set new frame ptr
    sub esp, 1048         ; allocate stack space
    mov eax, [ebp + 8]
    mov [esp + 4], eax     ; push arg (strcpy src)
    lea eax, [ebp - 1032]
    mov [esp], eax        ; push buffer (strcpy dest)
    call strcpy
    leave
    ret
```

gcc -S -m32

```
fun1: ; code with SSP (main function stays the same)
      ; NB: GS register points to per-CPU thread storage
      push ebp
      mov ebp, esp
      sub esp, 1064          ; use 16 bytes more this time
      mov eax, [ebp + 8]     ; fetch arg
      mov [ebp - 1052], eax  ; >> keep a copy in our frame
      mov eax, GS:[20]       ; >> set EAX=canary value
      mov [ebp - 12], eax    ; >> store near return address
      xor eax, eax
      mov eax, [ebp - 1052]  ; fetch local copy of arg
      mov [esp + 4], eax     ; push it
      ;
      lea eax, [ebp - 1036]
      mov [esp], eax        ; push buffer
      call strcpy
      mov edx, [ebp - 12]    ; >> EDX=canary from stack
      xor edx, GS:[20]      ; >> has it changed?
      je .L3                ; if it has, we'll abort
      call __stack_chk_fail

.L3:
      leave
      ret
```

The stack protection spots an overflow with 1026 characters:

```
$ gcc -m32 overflow.c -o overflow.out
$ ./overflow.out xxxx
$ ./overflow.out `perl -e 'print "x"x1025'`
*** stack smashing detected ***: ./overflow.out
terminated Aborted (core dumped)
```

Exercise. Try this example for yourself, compiling with/without protection, and stepping through it using gdb. Draw the stack layout in each case. Make up some more complex examples and try them out.

Security “arms race” and canaries

Attackers respond to new protection mechanisms by looking for vulnerabilities in those mechanisms (and other new vulnerabilities).

For example:

- ▶ Attack code/probing discovers a constant canary
 - e.g., canary is 0x0af237ab6, attacker fix: write that near return address
- ▶ Canary defense uses pseudorandom sequence
 - attacker learns sequence or discovers seed
- ▶ Canary defenses uses cryptographic PRNG
 - attacker finds where value is stored
 - finds another exploit to copy it